MediCore · Guion de defensa práctica

Pasos para responder en vivo lo que pide el documento: ejecutar el pipeline, provocar una falla, detener Redis o el consumidor, mostrar un hallazgo de ZAP, explicar el p95, recrear infraestructura y justificar la decisión GO / NO-GO.

Presentación

Diapositivas con notas por sección: https://claude.ai/artifact/1bNnrBdRYNPbqvP6RzfK5c

Antes de presentar

Hazlo con 30 minutos de anticipación. Docker Desktop debe estar encendido.

```
bash scripts/qa.sh up
```

```
bash scripts/qa.sh seed
```

```
bash scripts/qa.sh smoke
```

```
docker compose -p medicore-ci --env-file ci/.env.ci -f ci/docker-compose.ci.yml up -d
```

Deja abiertas estas páginas:

- Web: `http://127.0.0.1:8080`
- Jenkins: `http://127.0.0.1:8090`
- SonarQube: `http://127.0.0.1:9000`
- `reports/zap/zap-api.html`

El pipeline completo tarda demasiado para la exposición. Lleva abierta una corrida terminada y muestra en vivo solo la que falla.

1. Ejecutar el pipeline

1. Abre Jenkins, entra a **MediCore** y pulsa **Build with Parameters**.
2. Deja `SIMULAR_FALLO` sin marcar y pulsa **Build**.
3. Muestra las 18 etapas en la vista del trabajo.
4. Si te piden una corrida completa, muestra la última que terminó y sus reportes en **Artifacts**.

Qué decir: cada herramienta corre en contenedor, por eso el mismo pipeline funciona en cualquier equipo con Docker.

2. Provocar una prueba fallida

1. Pulsa **Build with Parameters**.

2. Marca `SIMULAR_FALLO` y pulsa **Build**.
3. La etapa 16 falla con el mensaje `Fallo simulado`.
4. Muestra que las etapas 17 y 18 quedaron omitidas.

Qué decir: una prueba crítica fallida, una vulnerabilidad crítica o un Quality Gate rechazado impiden el despliegue final.

3. Detener Redis

Inicia sesión en la web y detén Redis:

```
docker stop medicore-qa-redis-1
```

Qué ocurre: el inicio de sesión responde 503 y un token activo responde 401. No aparece un error 500.

Qué decir: el cache falla abierto y la sesión falla cerrado. Si no podemos comprobar una sesión, no entregamos datos clínicos.

Recupera el servicio:

```
docker start medicore-qa-redis-1
```

4. Detener el consumidor o RabbitMQ

Detén el Worker, que publica y consume eventos:

```
docker stop medicore-qa-worker-1
```

Registra un resultado de laboratorio en la web. Sigue guardado y su notificación queda en `Pending`.

Vuelve a iniciar el Worker:

```
docker start medicore-qa-worker-1
```

La notificación aparece como enviada. En el ensayo tardó segundos: ver `documentacion/evidencias/resiliencia-qa.md`.

Para detener el broker, usa `medicore-qa-rabbitmq-1` en lugar del Worker. El resultado también queda guardado.

Qué decir: el dato y su evento se guardan en la misma transacción. La entrega es al menos una vez, y el consumidor no duplica.

5. Mostrar un hallazgo de ZAP

1. Abre `reports/zap/zap-api-medico.html` y `reports/zap/cobertura-api.json`.
2. Muestra que no hay alertas altas.
3. Explica el defecto HZ-01: un carácter nulo en la búsqueda provocaba un error 500.
4. Muestra la corrección en `src/MediCore.Api/Security/InvalidTextExceptionHandler.cs`.
5. Muestra la prueba `Un_caracter_nulo_en_busqueda_o_en_datos_devuelve_400_y_no_500`.

Qué decir: ZAP se ejecutó con los 5 roles, con sesiones reales y datos válidos. Alcanzó 40 de las 45 operaciones del contrato, sin alertas altas ni errores 500. Las otras 5 están fuera a propósito. Antes el análisis era parcial porque usaba solo Administrador; lo corregimos y lo medimos.

6. Explicar el p95

El p95 es el tiempo que cumplen 95 de cada 100 peticiones. Refleja mejor la experiencia que el promedio, porque un promedio esconde a los usuarios más lentos.

Ejemplo: en Load, el p95 de reserva fue 8,9 ms. El 95 % de las reservas respondió en ese tiempo o menos, frente a un umbral de 1000 ms.

Límite: k6, la API y la base de datos corrieron en el mismo equipo, sin red real. Los números no representan una VM de Azure.

Hallazgo: la API respondía rápido, pero el perfil Spike dejó 14 405 eventos pendientes. Por eso también se mide la integridad después de la carga.

7. Recrear la infraestructura

Local, conservando los datos:

```
bash scripts/qa.sh down
```

```
bash scripts/qa.sh up
```

Ejecución de Terraform y Ansible sin Azure y sin costo (Ansible construye la aplicación, así que tarda varios minutos):

```
bash scripts/iac-simulacion.sh all
```

Qué decir: decidimos no usar recursos de pago, así que no hay despliegue real en Azure. Terraform recorrió aprovisionar, modificar y destruir con un proveedor simulado, y Ansible se aplicó en un Ubuntu desechable hasta dejar la aplicación funcionando. Eso prueba nuestro código, no la nube. La evidencia está en `documentacion/evidencias/iac-sin-costo.md`.

8. Restaurar un respaldo

Destruye QA y lo recupera desde una copia exportada. Tarda cerca de un minuto y medio:

```
QA_CONFIRM=medicore-qa bash scripts/simulacro-respaldo.sh
```

Qué decir: se borró la base y también el volumen de respaldos. Las 14 tablas volvieron idénticas. La pérdida máxima esperada es de 6 horas y las copias siguen en el mismo disco salvo que se exporten.

9. Justificar la decisión

La decisión está en la sección "Decisión GO / NO-GO" de `documentacion/13-informe-final.md`.

Separa lo que está probado de lo que falta. Si te preguntan por algo no ejecutado, dilo sin rodeos y muestra en qué lista de pendientes aparece.